របាយការណ៍បច្ចេកទេស — ការធ្វើតេស្តដែលមានការអនុញ្ញាត

php2shell — មគ្គុទ្ទេសក៍ធ្វើតេស្តសុវត្ថិភាព

CVE-2026-109618 / CVE-2026-109619

ភាសាខ្មែរ · លេខអារ៉ាប់ · សម្រាប់បរិស្ថាន lab នីមួយៗ និងការជួសជុលតាម branch

ទិន្នន័យបរិស្ថាន

ទីតាំងតេស្ត	p2s-lab = 127.0.0.1 (loopback)
OS	Debian bookworm (container)
SAPI	mod_php
PHP	8.4.26
Application	php2shell reference lab
Ports	8100–8106 បើកតែលើ 127.0.0.1
Profile ដែល RCE បញ្ជាក់	8.4.26 amd64 · 8.4.25 aarch64 · 8.5.10 aarch64 · 8.5.11 aarch64/amd64
ទិន្នន័យប្រភព	repo php2shell @ a3aa441 · catalog/vulnerable.json 2026-10-21
វគ្គឯកសារ	v1.1 · 28 តុលា 2026

សេចក្តីសង្ខេប

របាយការណ៍នេះពន្យល់ពីខ្សែសង្វាក់ php2shell, precondition ដែលត្រូវការ, របៀបវាយតម្លៃដោយមិនបង្កគ្រោះថ្នាក់ និងលំដាប់ជួសជុលតាម branch នីមួយៗ។ ចំណុចសំខាន់ពីរគឺ៖ បើគ្មាន LFI ខ្សែសង្វាក់ឈប់នៅជំហានទី 3, ហើយបើគ្មាន profile ដែល calibration រួចសម្រាប់ build នោះ dispatcher បដិសេធមិនរត់ទាល់តែសោះ។ RCE មិនត្រូវបានបញ្ជាក់ក្នុងទាំងពីរករណី។

ឯកសារនេះរត់តាម lab ក្នុងម៉ាស៊ីនរបស់អ្នក (127.0.0.1) ប៉ុណ្ណោះ។ ការធ្វើតេស្តលើប្រព័ន្ធរបស់អ្នកផ្សេងដោយគ្មានការអនុញ្ញាតជាលាយលក្ខណ៍អក្សរគឺខុសច្បាប់ ហើយខុសលក្ខខណ្ឌប្រើប្រាស់របស់ PHP2Shell។

មាតិកា

1. php2shell ជាអ្វី 3

2. តារាងជួសជុល 5

3. Preconditions 8

4. Assessment mode 11

5. ហេតុអ្វីត្រូវការ profile 13

6. mod_php vs mod_fcgid 16

7. សាងសង់ calibration container 18

8. រត់ calibration 21

9. ការពារ និងជួសជុល 23

10. Checklist ចុងក្រោយ 26

ជំពូកទាំង 10 ត្រូវបានរៀបចំតាមលំដាប់៖ ពីការយល់ពីខ្សែសង្វាក់ ទៅការវាយតម្លៃ និងការជួសជុល។

1 ជំពូក
php2shell ជាអ្វី

គោលបំណងនៃជំពូកនេះ

- យល់ពី CVE-2026-109618 និង CVE-2026-109619 នៅក្នុង PHP session upload progress។
- បែងចែកឱ្យបានច្បាស់រវាង vulnerability និង exploitability។
- មើលខ្សែសង្វាក់ 5 ជំហាន និងចំណុចពីរដែលវាឈប់៖ គ្មានផ្លូវ LFI ឬ គ្មាន profile ដែល calibration រួចសម្រាប់ build នោះ។

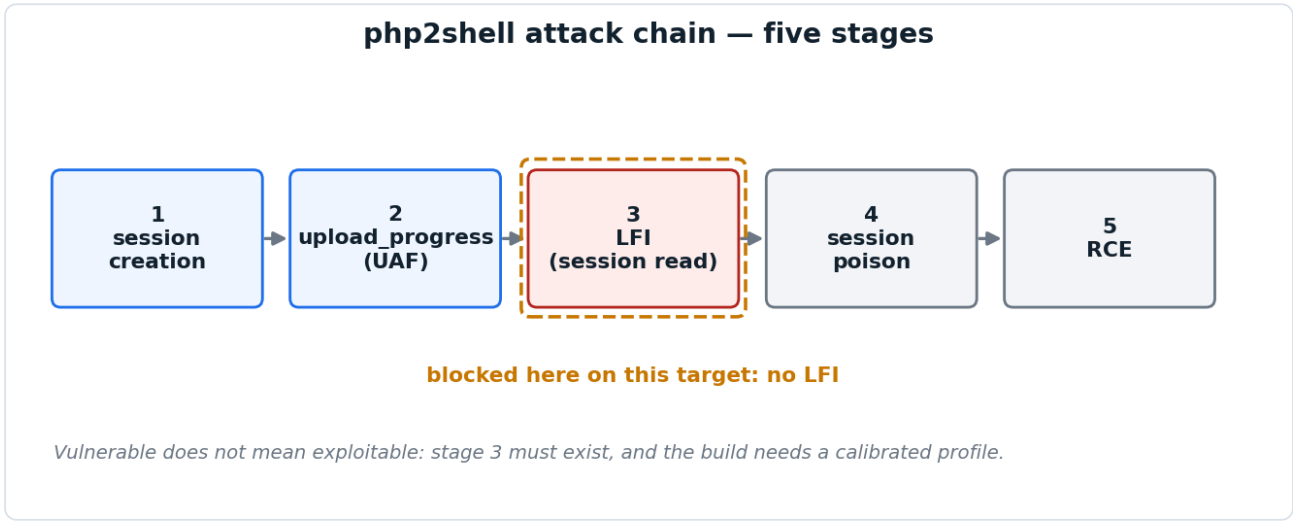
php2shell គឺជាឈ្មោះកញ្ចប់ឧបករណ៍ដែលគេប្រើដើម្បីធ្វើតេស្តភាពងាយរងគ្រោះពីរ គឺ CVE-2026-109618 និង CVE-2026-109619 នៅក្នុង PHP session upload progress។ ទាំងពីរជាបញ្ហា use-after-free ដែលកើតឡើងនៅពេល PHP គ្រប់គ្រង metadata នៃ upload progress ក្នុង session។ បញ្ហានេះអាចធ្វើឱ្យ memory ត្រូវបានបំផ្លាញ ប៉ុន្តែវាមិនមែនជា RCE ដោយខ្លួនឯងទេ។

ដើម្បីប្រែ memory corruption ទៅជា code execution ត្រូវការខ្សែសង្វាក់បន្ថែម៖ បង្កើត session, បញ្ជូន PHP_SESSION_UPLOAD_PROGRESS, រកផ្លូវ LFI ដើម្បីអាន session file, បន្ទាប់មកបញ្ចូល code ក្នុង session នោះ។ បើគ្មាន LFI ខ្សែសង្វាក់ឈប់នៅជំហានទី 3 ហើយ RCE មិនត្រូវបានបញ្ជាក់។ នេះជាចំណុចសំខាន់បំផុតនៃរបាយការណ៍នេះ។

> **ប្រយ័ត្ន**
> ភាពងាយរងគ្រោះមិនស្មើនឹងការអាច exploit បានទេ។ Version មានបញ្ហា តែបើ precondition មួយខ្វះ ខ្សែសង្វាក់មិនដំណើរការ។ ការប្រកាស RCE ដោយគ្មានភស្តុតាងគឺជាកំហុសវិភាគ។

នៅក្នុង lab របស់ repo ខ្សែសង្វាក់ RCE បានបញ្ជាក់លើ build ដែលមាន profile តែប៉ុណ្ណោះ៖ 8.4.26 amd64, 8.4.25 aarch64, 8.5.10 aarch64 និង 8.5.11 ទាំងពីរ architecture (proof ជា `uid=33(www-data)`)។ លើ 8.3.35 និង 8.3.33 មានតែ leak ជំហានទី 1–2 ដំណើរការ ចំណែកជំហានទី 3 មិន placement។ សម្រាប់ build ចាស់ដូចជា 7.4.33, 8.0.30, 8.1.34 គឺមានតែ symbol offsets ហើយមិនធ្លាប់ heap calibration — dispatcher បដិសេធមិនរត់ ព្រោះវាមិនប្រមាណ layout។

> **ចំណុចសំខាន់**
> ខ្សែសង្វាក់ឈប់ពីរកន្លែង៖ គ្មានផ្លូវ LFI (ជំហានទី 3) ឬ គ្មាន profile ដែល calibration រួចសម្រាប់ build នោះ (ជំហានទី 0)។ ក្នុងទាំងពីរករណី RCE មិនត្រូវបានបញ្ជាក់ ហើយរបាយការណ៍ត្រូវសរសេរបែបនេះ។



រូបភាព 1.1 — ខ្សែសង្វាក់វាយប្រហារ 5 ជំហាន និងចំណុចដែលរារាំងនៅលើគោលដៅនេះ

snippet — 01_check_php_version.sh

```
#!/usr/bin/env bash
# ពិនិត្យ version PHP និង header ដែលលេចចេញពីបរិស្ថានតេស្ត។
# មើល X-Powered-By និង php -v ក្នុង container។ ហានិភ័យ៖ ទាប។
set -u

TARGET="${1:-http://127.0.0.1:8100}"

echo "== headers =="
curl -skI "$TARGET/" | sed -n '1,20p'

echo "== X-Powered-By =="
curl -skI "$TARGET/" | grep -i '^x-powered-by' || echo "មិនឃើញ X-Powered-By"

echo "== version ពី lab (docker) =="
if command -v docker >/dev/null 2>&1; then
  docker exec "$(docker ps -q --filter publish=80 | head -1)" php -v 2>/dev/null || echo "គ្មាន container"
fi

echo "== version ពី lab (php2shell_labs.py) =="
python3 tools/php2shell_labs.py status 2>/dev/null | sed -n '1,20p' || true
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`php -v`	បង្ហាញ version PHP ក្នុង local console។ បើជា 8.0.30 ត្រូវផ្ទៀងផ្ទាត់ changelog បន្ថែម។
`X-Powered-By`	Header នេះលេចចេញ version។ ការបិទវាជួយកាត់បន្ថយ information leak។
`Server`	បង្ហាញ Apache និង OS។ មិនមែន vuln តែជាព័ត៌មានជំនួយសម្រាប់អ្នកវាយប្រហារ។
`rpm -q php`	បញ្ជាក់ NEVRA ដែលដំឡើង។ ត្រូវយកទៅធៀបនឹង erratum។

ចំណុចត្រួតពិនិត្យ: តើ version ដែល affected ដូចជា 8.0.30 មានន័យថាអាច exploit បានភ្លាមៗទេ?

ទេ។ Version នេះមានបញ្ហា upstream ប៉ុន្តែខ្សែសង្វាក់ទៅ RCE ត្រូវការ LFI, session file read និង profile ដែល heap calibration រួចសម្រាប់ build នោះ — 8.0.30 មានតែ symbol offsets ហើយគ្មាន calibration ដូច្នេះ dispatcher បដិសេធ។ ត្រូវផ្ទៀងផ្ទាត់ changelog និង preconditions មុនសន្និដ្ឋាន។

2 ជំពូក តារាងជួសជុល

គោលបំណងនៃជំពូកនេះ

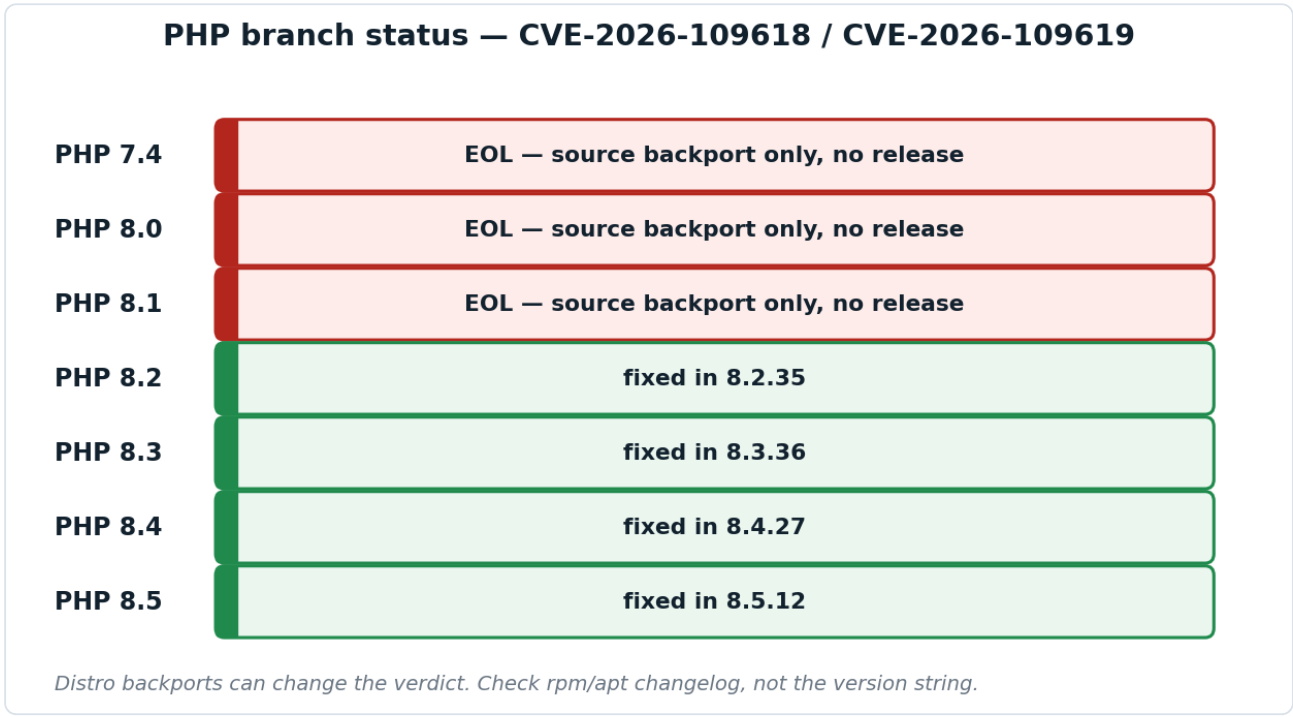
- អាន patch matrix តាម PHP branch 7.4 ដល់ 8.5 ជាមួយ release ដែល patched ជាក់លាក់។
- យល់ពីរបៀបដែល distro backport ផ្លាស់ប្តូរ verdict។
- ប្រើ rpm ឬ apt changelog ជាអ្នកកាន់អំណាច មិនមែន version string។

តារាងជួសជុលបង្ហាញថា branch EOL (7.4, 8.0, 8.1) មិនទទួល release ថ្មីទេ — មានតែ minimal source backport ក្នុង upstream ប៉ុណ្ណោះ។ Branch ដែលគាំទ្រមាន fix តាមលំដាប់៖ 8.2 ចាប់ពី 8.2.35, 8.3 ចាប់ពី 8.3.36, 8.4 ចាប់ពី 8.4.27, 8.5 ចាប់ពី 8.5.12។ តម្លៃទាំងនេះយកពី `catalog/vulnerable.json` (2026-10-21) ក្នុង repo។ តារាងនេះជាការណែនាំ មិនមែនការធានាថា package ជាក់លាក់មួយ patched ឬអត់ទេ។

Distro រក្សា version upstream ដូចជា 8.0.30 ហើយបន្ថែម patch ក្នុង release `-N.el9`។ ដូច្នេះត្រូវអាន rpm changelog និង dnf updateinfo មិនមែនមើល `php -v`។ បើ changelog មាន CVE-2026-109618 ឬ CVE-2026-109619 នោះ build ដែលដំឡើងមាន backport ហើយអ្នកអាចឈប់។ បើគ្មាន សូម upgrade ទៅ release ដែល patched តាម branch ខាងលើ។

> **ការពិត**
> version string 8.0.30 មិនប្រាប់ patch status ទេ។ Changelog និង erratum ជាអ្នកកាន់អំណាច។ ការសន្និដ្ឋានពី version តែម្នាក់ឯងអាចខុសទាំងសងខាង។

សម្រាប់ Debian និង Ubuntu ត្រូវពិនិត្យ apt changelog និង security tracker។ ឈ្មោះ package អាចខុសគ្នា ដូចជា php8.0 ឬ php8.1។ ការអាន changelog ត្រូវរកឃើញទាំងឈ្មោះ CVE និង release ដែលដំឡើង។



រូបភាព 2.1 — ស្ថានភាព PHP branch 8.0 ដល់ 8.5 សម្រាប់ CVE-2026-109618 និង CVE-2026-109619

snippet — 02_check_backport.sh

```
#!/usr/bin/env bash
# ផ្ទៀងផ្ទាត់ថា distro បាន backport CVE-2026-109618/109619 ឬអត់។
# មើល changelog របស់ package ដែលដំឡើង និង updateinfo។ ហានិភ័យ៖ ទាប (អានតែប៉ុណ្ណោះ)។
set -u

for cve in CVE-2026-109618 CVE-2026-109619; do
  echo "== $cve =="
  if command -v rpm >/dev/null 2>&1; then
    rpm -q --changelog php 2>/dev/null | grep -i "$cve" || echo "មិនឃើញក្នុង changelog php"
    dnf updateinfo info "$cve" 2>/dev/null | head -20 || true
  elif command -v apt >/dev/null 2>&1; then
    apt changelog php8.0 2>/dev/null | grep -i "$cve" || echo "មិនឃើញក្នុង apt changelog"
    apt list --upgradable 2>/dev/null | grep -i php || true
  fi
done
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`rpm -q php`	បង្ហាញ version-release ដែលដំឡើង។ ត្រូវកត់ត្រា NEVRA ទាំងស្រុង។
`rpm -q --changelog php | grep CVE`	បើឃើញ CVE នោះ build ដែលដំឡើងមាន backport។
`dnf updateinfo list cves`	បញ្ជី advisory ក្នុង repo។ បើគ្មាន CVE នេះ មិនមែនមានន័យថា patched ដោយស្វ័យប្រវត្តិទេ។
`php -v`	បង្ហាញ version upstream។ មិនគ្រប់គ្រាន់សម្រាប់សន្និដ្ឋាន patch status។
8.2 → 8.2.35	release ដំបូងដែល patched ក្នុង branch 8.2។
8.3 → 8.3.36	release ដំបូងដែល patched ក្នុង branch 8.3 (8.3.33/8.3.35 នៅ affected)។
8.4 → 8.4.27	release ដំបូងដែល patched ក្នុង branch 8.4 (8.4.25/8.4.26 RCE បញ្ជាក់)។
8.5 → 8.5.12	release ដំបូងដែល patched ក្នុង branch 8.5 (8.5.10/8.5.11 RCE បញ្ជាក់)។
7.4 / 8.0 / 8.1	EOL — គ្មាន release ថ្មី មានតែ minimal source backport។ ការពារដោយ config ជំនួស upgrade។

ចំណុចត្រួតពិនិត្យ: បើ changelog មាន CVE ទាំងពីរ តើត្រូវធ្វើអ្វីបន្ទាប់?

ឈប់រក exploit។ កត់ត្រា NEVRA, ថ្ងៃខែ, និងបន្ទាត់ changelog ជាភស្តុតាង។ បន្ទាប់មកផ្ទៀងផ្ទាត់ថា package ដែលដំឡើងគឺជា package ដែលកំពុងដំណើរការ មិនមែន package ចាស់ដែលនៅសល់ក្នុង cache។

3 ជំពូក Preconditions

គោលបំណងនៃជំពូកនេះ

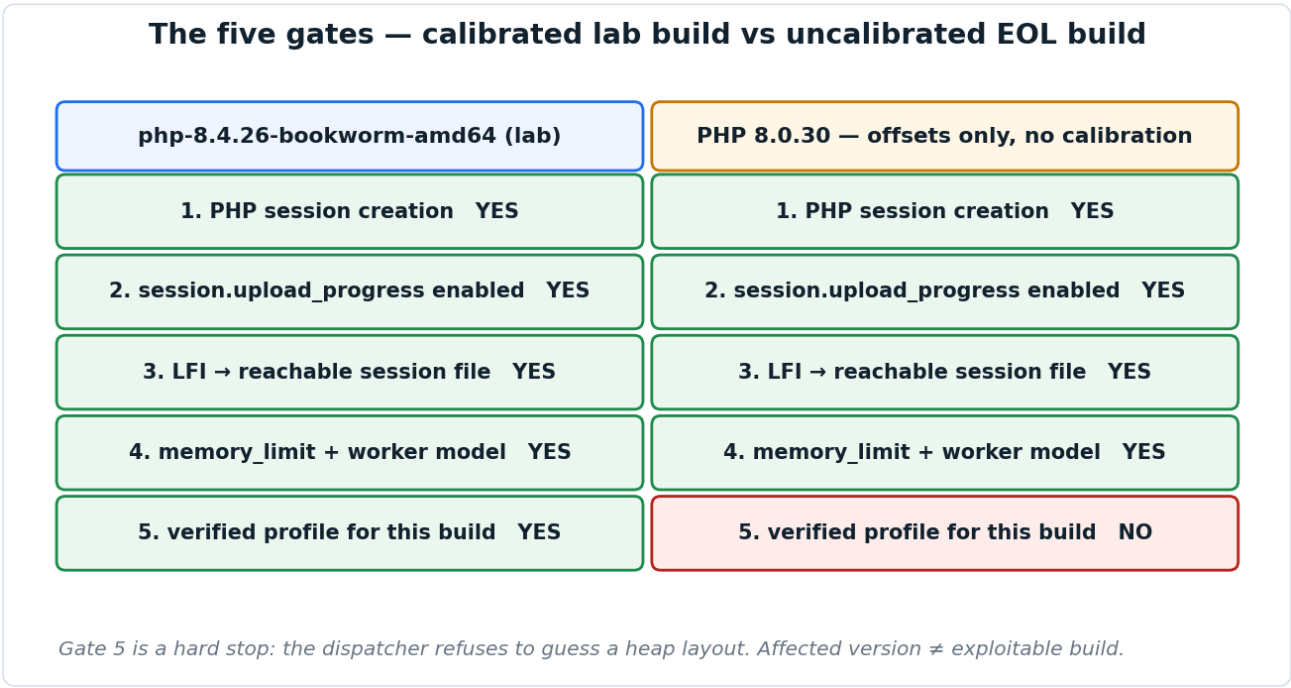
- រាយ precondition ទាំង 5 សម្រាប់ខ្សែសង្វាក់ php2shell។
- ផ្ទៀងផ្ទាត់ពួកវាដោយមិនបង្កគ្រោះថ្នាក់ដល់ production។
- យល់ពីហេតុអ្វី LFI ជាចំណុចខ្វះខាតដែលឈប់ខ្សែសង្វាក់។

ខ្សែសង្វាក់ត្រូវការ precondition យ៉ាងតិច 5 យ៉ាង៖ PHP ត្រូវបង្កើត session បាន, session.upload_progress ត្រូវបើក, ត្រូវមានផ្លូវ LFI ឬ session file read, memory_limit ត្រូវគ្រប់គ្រាន់សម្រាប់ heap grooming, និង worker model ត្រូវត្រូវនឹង profile។ បើមួយខ្វះ ការវាយតម្លៃត្រូវឈប់ ឬប្តូរទៅការជួសជុល។

ការផ្ទៀងផ្ទាត់ដោយសុវត្ថិភាពគឺធ្វើ multipart request តូចមួយជាមួយ PHP_SESSION_UPLOAD_PROGRESS ហើយមើល Set-Cookie។ បើមាន PHPSESSID ថ្មី នោះ session creation ដំណើរការ។ បន្ទាប់មកពិនិត្យ php -i ក្នុង local console សម្រាប់ session.upload_progress.enabled និង memory_limit។ មិនត្រូវ brute force ផ្លូវ LFI លើ production ទេ ព្រោះវាអាចបង្ក OOM ឬ kill worker។

> **ប្រយ័ត្ន**
> ការសាក probe ញឹកញាប់ ឬ request ធំៗ អាចធ្វើឱ្យ Apache worker គាំង និងបង្ក downtime។ ត្រូវធ្វើក្រៅម៉ោងធ្វើការ និងមានការអនុញ្ញាត។

លើ lab ដែលមាន profile ត្រូវគ្នា (8.4.26 amd64) precondition ទាំង 5 ឆ្លើយតប YES ហើយខ្សែសង្វាក់បញ្ជាក់ RCE ដោយ proof `uid=33(www-data)`។ លើ build ចាស់ដែលគ្មាន heap calibration (7.4.33, 8.0.30, 8.1.34, 8.2.34) ចំណុចទី 5 ជា NO — មិនមែនព្រោះវាមានសុវត្ថិភាពទេ ប៉ុន្តែព្រោះគ្មានភស្តុតាង។ ការសន្និដ្ឋានត្រូវផ្អែកលើ request, output និង profile មិនមែនការស្មាន។



រូបភាព 3.1 — Precondition 5 ចំណុច — lab ដែលមាន profile ត្រូវគ្នា (5 YES) បៀបធៀបនឹង build EOL ដែលគ្មាន heap calibration (4 YES, 1 NO)

snippet — 03_check_upload_progress.sh

```
#!/usr/bin/env bash
# ពិនិត្យថា PHP បង្កើត session ពេលធ្វើ PHP_SESSION_UPLOAD_PROGRESS។
# មើល Set-Cookie និង cookie PHPSESSID។ ហានិភ័យ៖ ទាប។
set -u

TARGET="${1:-http://127.0.0.1:8100}"
HEADERS=$(mktemp)
PAYLOAD=$(mktemp)
trap 'rm -f "$HEADERS" "$PAYLOAD"' EXIT
printf 'probe\n' > "$PAYLOAD"

curl -sk -D "$HEADERS" -o /dev/null \
  -H 'Cookie: PHPSESSID=p2sprobe' \
  -F 'PHP_SESSION_UPLOAD_PROGRESS=probe' \
  -F "file=@$PAYLOAD;filename=probe.txt" \
  "$TARGET/"

echo "== Set-Cookie =="
grep -i '^set-cookie' "$HEADERS" || echo "មិនឃើញ Set-Cookie"
echo "== PHPSESSID =="
grep -io 'PHPSESSID=[^;]*' "$HEADERS" || true
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`Set-Cookie: PHPSESSID`	បើមាន cookie ថ្មី session creation ដំណើរការ។ បើគ្មាន អាចមាន cache ឬ session ត្រូវបានបិទ។
`session.upload_progress.enabled`	ត្រូវជា On សម្រាប់ខ្សែសង្វាក់។ បើ Off ជំហានទី 2 មិនដំណើរការ។
`memory_limit`	128M ជាតម្លៃធម្មតា។ តម្លៃតូចពេកអាចធ្វើឱ្យ heap grooming បរាជ័យ។
`HTTP status`	200 ឬ 400 អាចជាធម្មតា។ ត្រូវមើល body និង header មិនមែនតែ status។

ចំណុចត្រួតពិនិត្យ: ហេតុអ្វី LFI សំខាន់ម្ល៉េះ?

ព្រោះ session file មាន metadata ដែលអ្នកវាយប្រហារគ្រប់គ្រងបាន។ បើគ្មានផ្លូវអានវា ការសរសេរ code ក្នុង session មិនអាចប្រែទៅជា execution បានទេ។ ដូច្នេះ LFI ជាស្ពានរវាង memory corruption និង RCE។

ជំពូក 4

Assessment mode

គោលបំណងនៃជំពូកនេះ

- ដឹងថា php2shell_assess.py ពិនិត្យអ្វីខ្លះ និងអ្វីដែលវាមិនធ្វើ។
- អាន verdict ឱ្យបានត្រឹមត្រូវ និងមិនច្រឡំជាមួយ RCE proof។
- ប្រើ assessment ជាជំហានដំបូងមុន calibration។

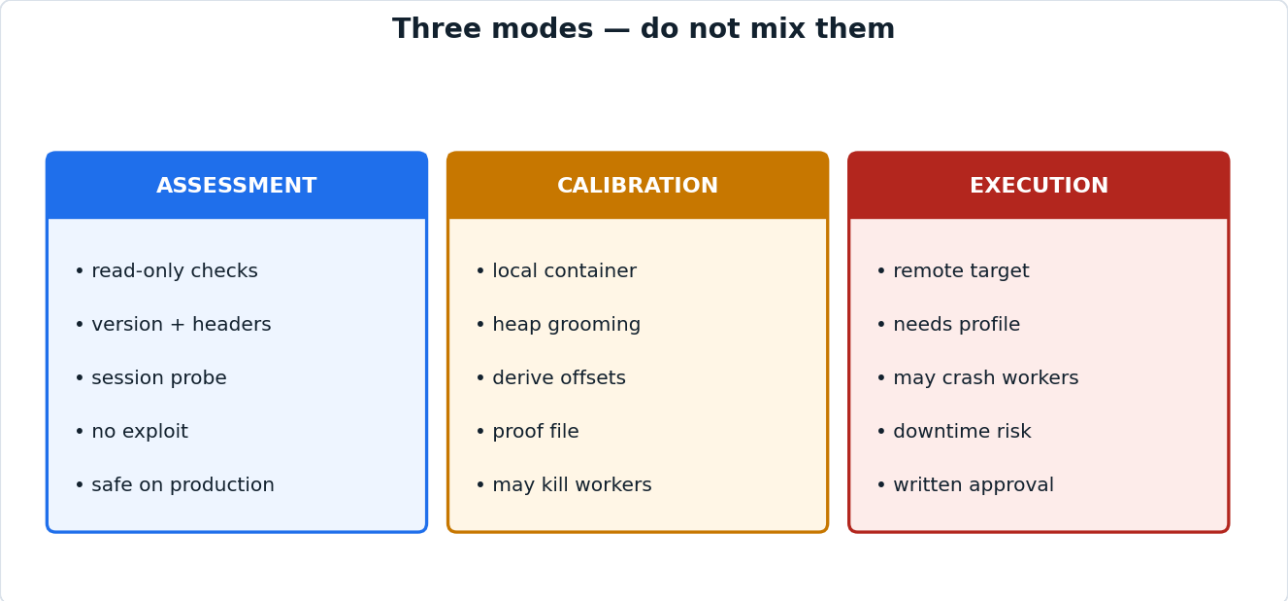
Assessment mode គឺជាការពិនិត្យបែប read-only។ ឧបករណ៍ពិនិត្យ version, header, session behavior, upload progress និងសញ្ញានៃ LFI។ វាមិនផ្ញើ payload ដែលបំផ្លាញ memory មិនរត់ exploit និងមិនបង្កើត proof នៃ code execution ទេ។ ដូច្នេះវាអាចប្រើលើ production បាន ប៉ុន្តែនៅតែត្រូវមានការអនុញ្ញាត។

Verdict មានបីបែប៖ vulnerable, not vulnerable និង inconclusive។ vulnerable មានន័យថា precondition គ្រប់គ្រាន់ត្រូវបានរកឃើញ មិនមែនមានន័យថា RCE បានជោគជ័យទេ។ inconclusive មានន័យថាខ្វះភស្តុតាង ដូចជាមិនអាចផ្ទៀងផ្ទាត់ LFI បាន។ ត្រូវអានហេតុផលនៅក្នុង JSON មិនមែនតែពាក្យ verdict។

គន្លឹះ

ប្រើ assessment ដើម្បីចម្រាញ់គោលដៅ មុននឹងចំណាយពេលលើ calibration។ បើ assessment បញ្ជាក់ថាគ្មាន LFI សូមឈប់ ហើយជួសជុល។

លើ lab 8.4.26 assessment រាយពិន្ទុ PASS តាម precondition ទាំងអស់ ហើយបញ្ជាក់ថាមាន profile ដែល verification រួច។ លើ build EOL ដូចជា 8.0.30 វារាយ inconclusive — មិនមែនដោយសារវាមានសុវត្ថិភាព ប៉ុន្តែដោយសារគ្មានភស្តុតាង heap calibration។ ការរាយការណ៍លើសពីអ្វីដែលវាវាស់បានគឺជាកំហុសធ្ងន់ធ្ងរ។



រូបភាព 4.1 — ប្រៀបធៀប assessment, calibration និង execution — កុំលាយគ្នា

snippet — 04_assess_target.sh

```
#!/usr/bin/env bash
# ហៅ php2shell_assess.py លើ lab ដើម្បីវាយតម្លៃដោយមិនរត់ exploit។
# មើល verdict និងហេតុផល។ លើ lab loopback មិនត្រូវការ --allow-remote ទេ។ ហានិភ័យ៖ ទាប។
set -u

TARGET="${1:-http://127.0.0.1:8100}"

python3 tools/php2shell_assess.py \
  --target "$TARGET" \
  --php-version 8.4.26 \
  --output "assess-$(date +%Y%m%d-%H%M%S).json"

# បើគោលដៅមិនមែន loopback ត្រូវបន្ថែមទាំងអស់នេះ (repo បដិសេធមិនប្រមាណ)៖
#   --allow-remote --arch amd64   ឬ --allow-remote --profile profiles/8.4.26-bookworm-amd64.json
# ហើយមានតែប្រព័ន្ធដែលអ្នកជាម្ចាស់ ឬមានការអនុញ្ញាតជាលាយលក្ខណ៍អក្សរប៉ុណ្ណោះ។
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`verdict`	vulnerable, not vulnerable ឬ inconclusive។ ត្រូវអានហេតុផលជាមួយ។
`php_version`	បញ្ជាក់ version ពិតរបស់ build មុនជ្រើស profile។ បើខុស ត្រូវកែ config រួចរត់ឡើងវិញ។
`lfi`	បើ false ខ្សែសង្វាក់ឈប់។ នេះជាលទ្ធផលសំខាន់បំផុត។
`session`	បង្ហាញថា upload progress បង្កើត session បាន ឬអត់។

ចំណុចត្រួតពិនិត្យ: តើ assessment អាចបញ្ជាក់ RCE បានទេ?

ទេ។ វាបញ្ជាក់បានតែលក្ខខណ្ឌដែលអាចអនុញ្ញាតឱ្យមាន exploit។ RCE proof ត្រូវការ calibration និង execution ដែលមានការអនុញ្ញាតដាច់ដោយឡែក។

ជំពូក 5

ហេតុអ្វីត្រូវការ profile

គោលបំណងនៃជំពូកនេះ

- យល់ពី heap grooming, offsets, libc និង allocator។
- ដឹងថាហេតុអ្វី profile Debian + mod_php មិនដំណើរការលើ AlmaLinux + mod_fcgid។
- រៀបចំ calibration ដើម្បីបង្កើត profile ថ្មី។

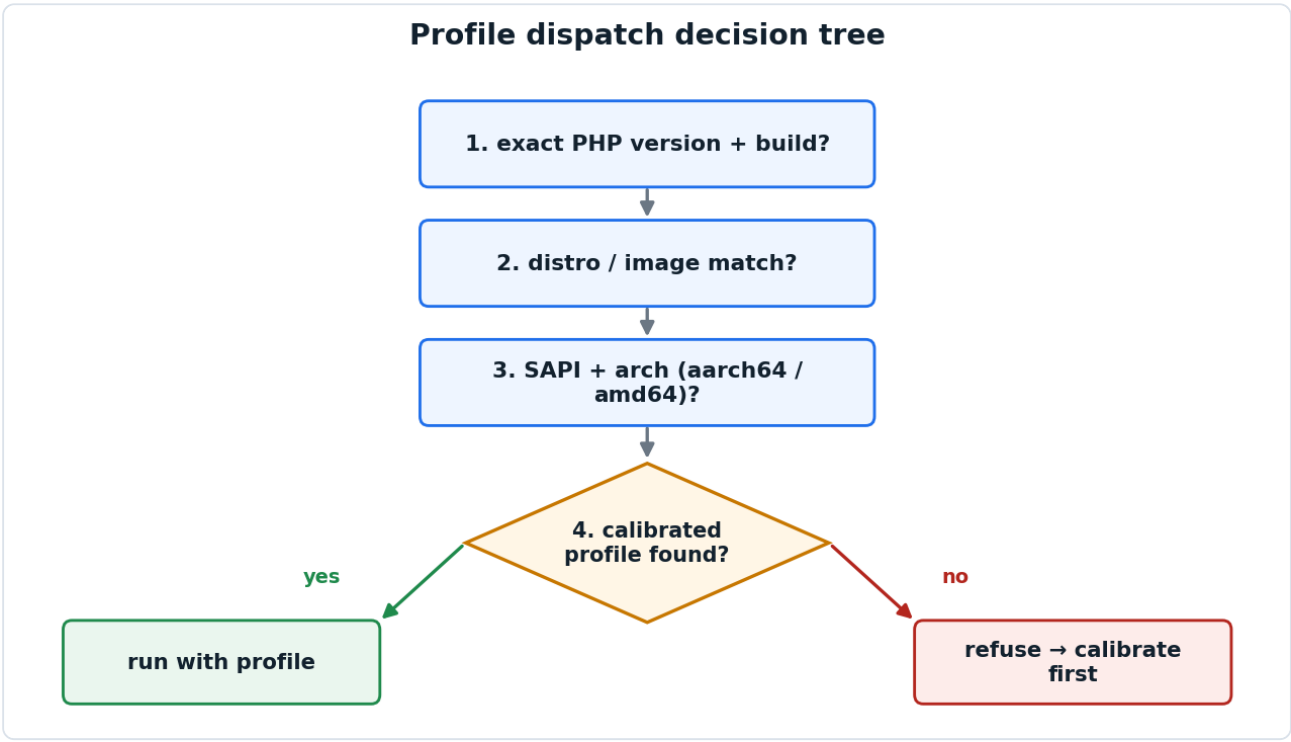
Exploit សម្រាប់ memory corruption ត្រូវពឹងលើ layout នៃ heap និង address នៃ function ឬ structure ជាក់លាក់។ Heap grooming គឺជាការរៀបចំ allocation និង free ដើម្បីឱ្យ object ដែលរងគ្រោះស្ថិតនៅទីតាំងដែលគ្រប់គ្រងបាន។ Offsets អាចផ្លាស់ប្តូរតាម version របស់ PHP, libc, compiler flags និង SAPI។

Profile ដែលគេចេញផ្សាយជាធម្មតាសម្រាប់ Debian bookworm និង mod_php។ វាមិនអាចយកមកប្រើលើ AlmaLinux 9 និង mod_fcgid បានទេ ព្រោះ process model, allocator និង session path ខុសគ្នា។ ការប្រើ profile ខុសអាចធ្វើឱ្យ exploit បរាជ័យ ឬធ្ងន់ជាងនោះគឺ kill worker និងបង្ក downtime។

> **ការពិត**
> Profile គឺ build-specific។ ត្រូវ re-derive រាល់ពេលមាន patch, ការផ្លាស់ PHP stream ឬការផ្លាស់ SAPI។ ការសន្មតថា profile ចាស់នៅតែប្រើបានគឺជាហានិភ័យ។

ដូច្នេះជំហានបន្ទាប់គឺសាងសង់ container ដែលជិត target បំផុត ហើយរត់ calibration នៅលើ loopback។ បន្ទាប់ពីទទួលបាន proof file និង profile ទើបពិចារណា execution លើ remote ដោយមានការអនុញ្ញាតជាលាយលក្ខណ៍អក្សរ។

dispatcher មិនប្រមាណ layout ទេ។ បើ build នោះមិនមាន profile ដែល calibration រួច វាបដិសេធការរត់ — នេះជាមូលហេតុដែល build EOL ដូចជា 8.0.30 រាយការណ៍ inconclusive ទោះ preconditions ផ្សេងទៀត PASS ក៏ដោយ។ សម្រាប់គោលដៅខាងក្រៅ ត្រូវបញ្ជាក់ `--arch` ឬ `--profile` ដោយផ្ទាល់ ហើយឈ្មោះ architecture ប្រើ `aarch64` និង `amd64`។



រូបភាព 5.1 — ដើមឈើសម្រេចចិត្តសម្រាប់ជ្រើស profile តាម version, distro និង SAPI

snippet — 06_run_calibration.sh

```
#!/usr/bin/env bash
# រត់ calibrate.py លើ container local តាម loopback ដើម្បីបង្កើត profile។
# មើល proof file និងថា target ជា 127.0.0.1:8100។ ហានិភ័យ៖ មធ្យម (អាច kill worker)។
set -euo pipefail

python3 calibrate.py \
  --target http://127.0.0.1:8100 \
  --arch amd64 \
  --php-version 8.0.30 \
  --sapi mod_fcgid \
  --distro almalinux-9 \
  --proof-file ./proof.json \
  --output ./profile-alma9-8030.json
```

file — profiles/ — inventory ពី repo

```
# profiles/*.json — repo a3aa441
# offsets គិតពី libphp.so load base (ASLR បូរ base មិនបូរ offset)
PHP      arch     zif_system zval_ptr_dtor htdata  N    forge T    RCE
7.4.33   aarch64  0x309640   0x3f6024      -       -    -     -    no
8.0.30   aarch64  0x31ef70   0x40ddb0      -       -    -     -    no
8.1.34   aarch64  0x341e08   0x42e7a0      -       -    -     -    no
8.2.34   aarch64  0x36ff48   0x4606c0      -       0    0     406  no
8.3.33   mod_php  0x377248   0x4737e0      -       815  63    406  no
8.3.35   aarch64  0x377428   0x46fcc0      -       841  63    406  no
8.4.25   aarch64  0x42fa30   0x637ae0      0x690d8 841  63    406  yes
8.4.26   amd64    0x465370   0x65eb60      0x690d8 841  79    406  yes
8.5.10   aarch64  0x52e130   0x75d360      0x690d8 841  63    406  yes
8.5.11   aarch64  0x5314f0   0x761b80      0x690d8 841  63    406  yes
8.5.11   amd64    0x5996e0   0x7b47a0      0x690d8 841  63    406  yes

# 7.4.33 / 8.0.30 / 8.1.34 → offsets only, heap not calibrated
# 8.2.34 → calibration failed (leak shape differs)
# 8.3.33 / 8.3.35 → stage 1-2 reproduce, stage 3 not placed (fix 8.3.36)
# 8.4.25 → probabilistic stage 3, use default retries (fix 8.4.27)
# 8.4.26 → reference build; 8.5.10/8.5.11 verified (fix 8.5.12)
# បន្ថែម build ថ្មី៖ ដាក់ក្នុង BUILDS នៃ tools/php2shell_matrix.py រួចរត់ factory
# aarch64 និង amd64 នៃ version ដូចគ្នា មាន offset និង heap layout ខុសគ្នា
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`distro`	AlmaLinux 9 ខុសពី Debian bookworm។ Profile ត្រូវផ្គូផ្គង distro។
`sapi`	mod_fcgid មាន process model ខុសពី mod_php។ Offsets អាចខុស។
`arch`	`aarch64` និង `amd64` នៃ version ដូចគ្នា មាន offsets ខុសគ្នា (ឧ. 8.5.11 មាន profile ពីរ)។ ត្រូវផ្ទៀងផ្ទាត់ uname -m។
`libc`	Version libc ប៉ះពាល់ address និង gadget។ ត្រូវផ្ទៀងផ្ទាត់ក្នុង container។
`profile match`	បើមិន match ត្រូវ derive ថ្មី មិនត្រូវរត់ remote ទេ។

ចំណុចត្រួតពិនិត្យ: ហេតុអ្វី profile Debian មិនអាចប្រើលើ AlmaLinux បាន?

ព្រោះ heap layout, libc, compiler flags, SAPI និង session path ខុសគ្នា។ Exploit ដែលពឹងលើ offset ជាក់លាក់នឹងបរាជ័យ ឬបង្កគ្រោះថ្នាក់ដល់ worker។ ត្រូវ calibration លើបរិស្ថានដែលដូច target។

6 ជំពូក mod_php vs mod_fcgid

គោលបំណងនៃជំពូកនេះ

- យល់ពី process model របស់ mod_php និង mod_fcgid។
- មើលភាពខុសគ្នានៃ memory layout និង session save_path។
- ផ្ទៀងផ្ទាត់ SAPI ដែល target កំពុងប្រើ។

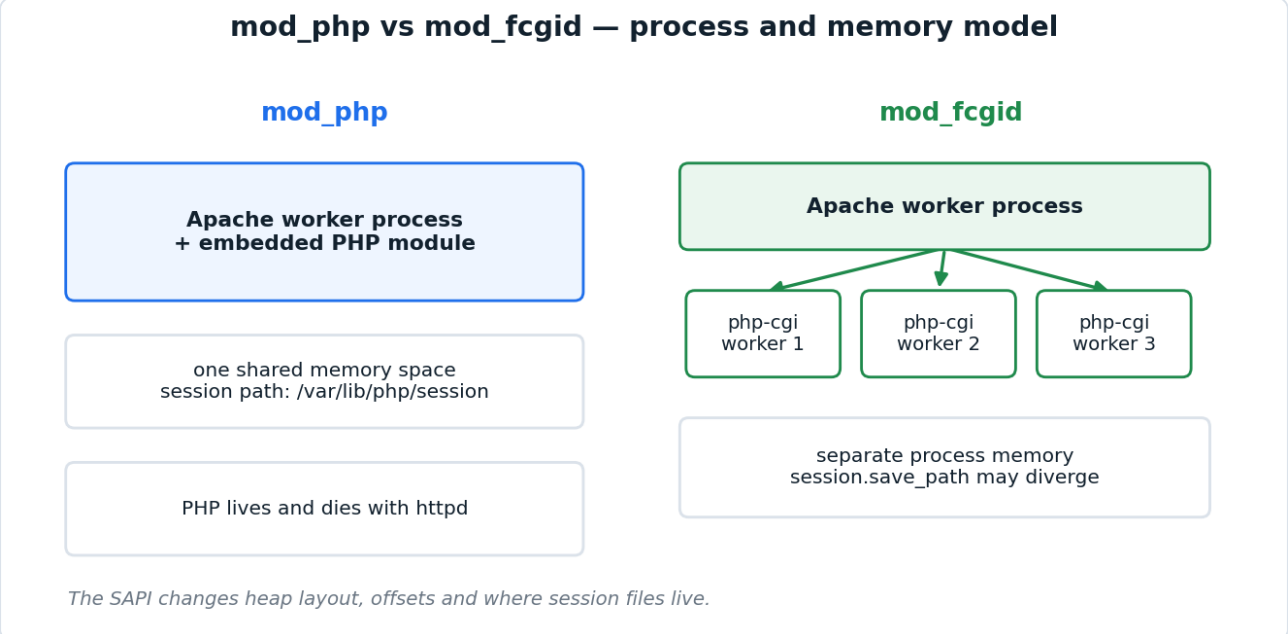
mod_php ដាក់ PHP នៅក្នុង process របស់ Apache ដោយផ្ទាល់។ Request និង PHP ចែករំលែក memory space តែមួយ ហើយ module ត្រូវបានផ្ទុកចូល Apache។ ការកំណត់ session និង memory អាចខុសពី SAPI ផ្សេង។ នេះជាមូលហេតុដែល profile mod_php មិនអាចផ្ទេរទៅ mod_fcgid ដោយស្វ័យប្រវត្តិ។

mod_fcgid ប្រើ process ដាច់ដោយឡែកសម្រាប់ php-cgi ហើយ Apache ទាក់ទងតាម FastCGI។ រាល់ worker មាន memory space ផ្ទាល់ខ្លួន។ Session save_path អាចជា /var/lib/php/sessions ឬ path ផ្សេង អាស្រ័យ config។ នៅលើ target នេះ mod_fcgid 2.3.9 កំពុងដំណើរការ ដូច្នេះត្រូវប្រើ profile សម្រាប់ mod_fcgid។

ចំណុចសំខាន់

SAPI ផ្លាស់ប្តូរ heap layout និង session path។ ការមើលតែ PHP version មិនគ្រប់គ្រាន់សម្រាប់ការសម្រេចថា profile ត្រូវគ្នាឬអត់ទេ។

ដើម្បីផ្ទៀងផ្ទាត់ សូមមើល httpd -M សម្រាប់ fcgid, php -i សម្រាប់ session.save_path និង process list សម្រាប់ php-cgi។ បើឃើញ libphp.so ជំនួស fcgid នោះ target កំពុងប្រើ mod_php ហើយការវាយតម្លៃត្រូវផ្លាស់។ ការកត់ត្រាភស្តុតាងទាំងនេះជួយឱ្យរបាយការណ៍អាចផ្ទៀងផ្ទាត់បាន។



រូបភាព 6.1 — ដ្យាក្រាម process របស់ mod_php និង mod_fcgid ជាមួយភាពខុសគ្នានៃ memory និង session

snippet — 07_verify_container.sh

```
#!/usr/bin/env bash
# ផ្ទៀងផ្ទាត់ថា container មាន PHP 8.0.30, mod_fcgid និង session.save_path ត្រឹមត្រូវ។
# មើល php -v, httpd -M និង php -i។ ហានិភ័យ៖ ទាប។
set -u

docker exec p2s-calib php -v
docker exec p2s-calib httpd -M 2>/dev/null | grep -i fcgid || echo "មិនឃើញ mod_fcgid"
docker exec p2s-calib php -i | grep -E 'session.save_path|session.upload_progress|memory_limit'
docker exec p2s-calib httpd -v
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`httpd -M | grep fcgid`	បើឃើញ fcgid_module target ប្រើ mod_fcgid។ បើឃើញ php_module វាជា mod_php។
`php -i | grep session.save_path`	បញ្ជាក់ path ពិតរបស់ session។ ត្រូវផ្គូផ្គងជាមួយ profile។
`ps aux | grep php-cgi`	ឃើញ worker ដាច់ដោយឡែក = mod_fcgid។ គ្មាន = អាចជា mod_php ឬ FPM។
`php -v`	បញ្ជាក់ version តែមិនបញ្ជាក់ SAPI។ ត្រូវមើលពីរបៀបផ្សេងទៀត។

ចំណុចត្រួតពិនិត្យ: តើ mod_fcgid មាន session.save_path ដូច mod_php ជានិច្ចទេ?

ទេ។ វាអាចខុសគ្នាតាម php.ini, pool config ឬ container។ ត្រូវពិនិត្យដោយផ្ទាល់ជាមួយ php -i និងកត់ត្រាលទ្ធផល។ ការសន្មតថាដូចគ្នាអាចធ្វើឱ្យ calibration បរាជ័យ។

ជំពូក 7

សាងសង់ calibration container

គោលបំណងនៃជំពូកនេះ

- សាងសង់ AlmaLinux 9 + PHP 8.0.30 + Apache 2.4.62 + mod_fcgid។
- Pin version និងផ្ទៀងផ្ទាត់ថា container ដូច target។
- ដឹងពីរបៀបកែ បើ package pin មិនមានក្នុង repo។

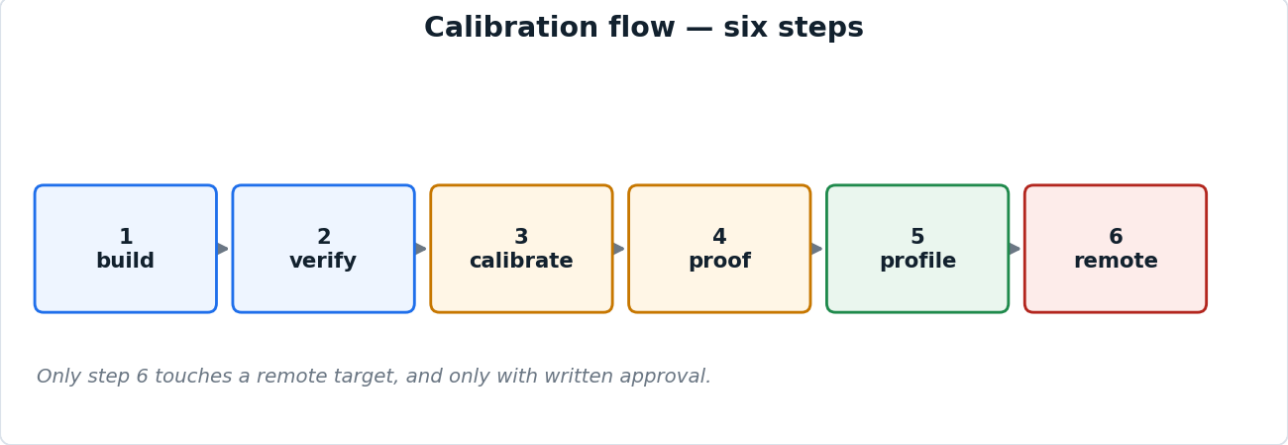
Calibration container ត្រូវជិត target បំផុត៖ AlmaLinux 9, PHP 8.0.30, Apache 2.4.62 និង mod_fcgid 2.3.9។ ការប្រើ image ផ្សេង ឬ PHP version ផ្សេង នឹងធ្វើឱ្យ offsets និង heap layout ខុស។ Dockerfile ខាងក្រោមដំឡើង EPEL និង Remi, បើក php:remi-8.0 ហើយ pin PHP 8.0.30។

បន្ទាប់ពី build ត្រូវផ្ទៀងផ្ទាត់ថា container មាន version ត្រឹមត្រូវ, mod_fcgid ត្រូវបានផ្ទុក, session.save_path ជា /var/lib/php/sessions និង session.upload_progress.enabled=On។ បើខុស ត្រូវកែ Dockerfile មុនរត់ calibration។ ការផ្ទៀងផ្ទាត់នេះជាការការពារកុំឱ្យ profile ខុសត្រូវបានបង្កើត។

ប្រយ័ត្ន

បើ dnf រក package 8.0.30 មិនឃើញ សូមប្រើ dnf --showduplicates list php ដើម្បីរក NEVRA ដែលមាន ហើយកែលេខ release ក្នុង Dockerfile។ កុំបន្ថយទៅ version ផ្សេងដោយគ្មានការកត់ត្រា។

Container គួររត់លើ port 8100 តាម loopback ប៉ុណ្ណោះ។ មិនត្រូវបើក port នេះទៅ internet ទេ។ បន្ទាប់ពីបញ្ចប់ការងារ ត្រូវលុប container និង image ដើម្បីកាត់បន្ថយ attack surface និងធនធាន។



រូបភាព 7.1 — លំដាប់ 6 ជំហានពី build ដល់ remote execution

snippet — 05_build_calibration.sh

```
#!/usr/bin/env bash
# សាងសង់ container AlmaLinux 9 + PHP 8.0.30 + Apache 2.4.62 + mod_fcgid។
# មើលថា build ជោគជ័យ និង container ចាប់ផ្តើមលើ port 8100។ ហានិភ័យ៖ ទាប (local)។
set -euo pipefail

cd "$(dirname "$0")/.."
docker build -f Dockerfile.calib -t p2s-alma9-8030 .
docker rm -f p2s-calib 2>/dev/null || true
docker run -d --name p2s-calib -p 8100:80 p2s-alma9-8030
docker ps --filter name=p2s-calib
```

snippet — 07_verify_container.sh

```
#!/usr/bin/env bash
# ផ្ទៀងផ្ទាត់ថា container មាន PHP 8.0.30, mod_fcgid និង session.save_path ត្រឹមត្រូវ។
# មើល php -v, httpd -M និង php -i។ ហានិភ័យ៖ ទាប។
set -u

docker exec p2s-calib php -v
docker exec p2s-calib httpd -M 2>/dev/null | grep -i fcgid || echo "មិនឃើញ mod_fcgid"
docker exec p2s-calib php -i | grep -E 'session.save_path|session.upload_progress|memory_limit'
docker exec p2s-calib httpd -v
```

file — Dockerfile.calib

```
# ប្រើ AlmaLinux 9 ដើម្បីឱ្យជិត target ពិត។
FROM almalinux:9

# ដំឡើង EPEL និង Remi សម្រាប់ PHP 8.0។
RUN dnf install -y epel-release && \
    dnf install -y https://rpms.remirepo.net/enterprise/remi-release-9.rpm && \
    dnf module reset php -y && \
    dnf module enable php:remi-8.0 -y

# ដំឡើង Apache, mod_fcgid និង PHP 8.0.30 ដែល pin version។
RUN dnf install -y httpd mod_fcgid \
      php-8.0.30 php-cli-8.0.30 php-fpm-8.0.30 php-common-8.0.30 && \
    dnf clean all

# កំណត់ mod_fcgid ឱ្យហៅ php-cgi និងបង្កើត session directory។
RUN printf '%s\n' \
      '<IfModule mod_fcgid.c>' \
      '  AddHandler fcgid-script .php' \
      '  FcgidWrapper /usr/bin/php-cgi .php' \
      '  FcgidMaxRequestLen 67108864' \
      '</IfModule>' \
      '<Directory /var/www/html>' \
      '  Options +ExecCGI' \
      '  AllowOverride All' \
      '  Require all granted' \
      '</Directory>' \
      > /etc/httpd/conf.d/fcgid-php.conf && \
    mkdir -p /var/lib/php/sessions && \
    chown -R apache:apache /var/lib/php/sessions

# កំណត់ PHP ឱ្យប្រើ session path តែមួយ និងបើក upload progress។
RUN printf '%s\n' \
      'session.save_path = "/var/lib/php/sessions"' \
      'session.upload_progress.enabled = On' \
      'memory_limit = 128M' \
      > /etc/php.d/99-p2s.ini

# បង្កើត index.php តូចមួយដែលចាប់ផ្តើម session។
RUN printf '%s\n' '<?php session_start(); echo "p2s-calib"; ?>' > /var/www/html/index.php && \
    chown apache:apache /var/www/html/index.php

# បើក port 80 និងរត់ httpd នៅ foreground។
EXPOSE 80
CMD ["/usr/sbin/httpd", "-D", "FOREGROUND"]
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`docker build -f Dockerfile.calib`	បង្កើត image។ បើ package pin បរាជ័យ ត្រូវកែ NEVRA។
`docker run -p 8100:80`	រត់ container តាម loopback។ មិនត្រូវបើកឱ្យខាងក្រៅ។
`docker exec php -v`	ត្រូវបង្ហាញ PHP 8.0.30។ បើខុស ការផ្ទៀងផ្ទាត់បរាជ័យ។
`httpd -M | grep fcgid`	ត្រូវឃើញ fcgid_module។ បើគ្មាន container មិនដូច target។

ចំណុចត្រួតពិនិត្យ: ហេតុអ្វីត្រូវ pin PHP 8.0.30 ក្នុង container?

ព្រោះ heap layout និង offsets អាចផ្លាស់ប្តូរតាម version តូចៗ។ បើ container ប្រើ 8.0.31 ឬ 8.0.29 profile ដែលបានមកអាចមិនត្រូវនឹង target។ ការតម្លើង version ឱ្យដូច target ជួយឱ្យ proof មានអត្ថន័យ។

8 ជំពូក
រត់ calibration

គោលបំណងនៃជំពូកនេះ

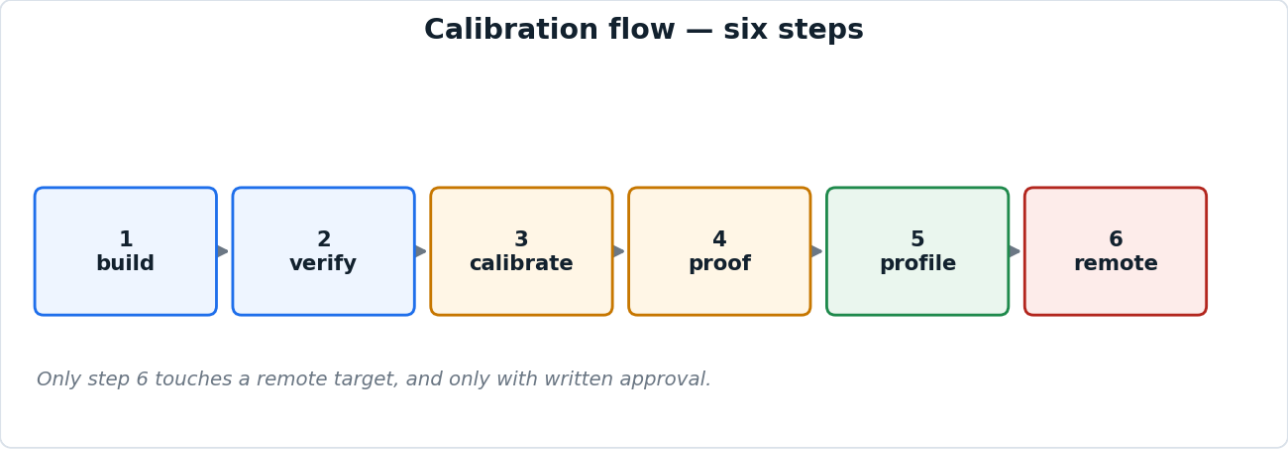
- រត់ calibrate.py តែលើ loopback 127.0.0.1:8100។
- អាន output និង proof file ឱ្យបានត្រឹមត្រូវ។
- ដឹងពីហានិភ័យនៃការរត់ execution លើ remote។

Calibration ត្រូវរត់តែលើ container local តាម 127.0.0.1:8100។ ការប្រើ --arch amd64 ធានាថា gadget និង offset ត្រូវនឹង architecture របស់ target។ បើអ្នកប្រើ ARM ឬ container ផ្សេង architecture លទ្ធផលនឹងមិនអាចយកទៅប្រើបាន។ ត្រូវផ្ទៀងផ្ទាត់ uname -m ក្នុង container មុន។

Output របស់ calibrate.py បង្ហាញជំហាន heap grooming, ការសាកល្បង offset និងលទ្ធផល proof។ Proof file ជាភស្តុតាងថា exploit ដំណើរការក្នុងបរិស្ថាន calibration។ វាមិនមែនជាភស្តុតាងថា target ពិតអាច exploit បានទេ បើ preconditions លើ target ខ្ះ។ ត្រូវអានជាមួយ assessment និង LFI result។

ប្រយ័ត្ន
ការរត់ execution លើ remote អាច kill Apache worker និងបង្ក downtime។ កុំរត់ក្នុងម៉ោងធ្វើការ។
ត្រូវមានការអនុញ្ញាតជាលាយលក្ខណ៍អក្សរ និងផែនការ rollback។

បើ calibration បរាជ័យ កុំបន្តទៅ remote ដោយស្មាន។ ត្រូវពិនិត្យថា container ត្រូវនឹង target, session path ត្រូវគ្នា, mod_fcgid ដំណើរការ និង PHP version ត្រឹមត្រូវ។ បើនៅតែបរាជ័យ សូមកត់ត្រាថា RCE មិនត្រូវបានបញ្ជាក់ ហើយបន្តទៅការជួសជុល។



រូបភាព 8.1 — លំដាប់ calibration និងចំណុចដែលត្រូវឈប់មុន remote execution

snippet — 06_run_calibration.sh

```
#!/usr/bin/env bash
# រត់ calibrate.py លើ container local តាម loopback ដើម្បីបង្កើត profile។
# មើល proof file និងថា target ជា 127.0.0.1:8100។ ហានិភ័យ៖ មធ្យម (អាច kill worker)។
set -euo pipefail

python3 calibrate.py \
  --target http://127.0.0.1:8100 \
  --arch amd64 \
  --php-version 8.0.30 \
  --sapi mod_fcgid \
  --distro almalinux-9 \
  --proof-file ./proof.json \
  --output ./profile-alma9-8030.json
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`--target 127.0.0.1:8100`	ត្រូវជា loopback។ បើជា IP ខាងក្រៅ អ្នកកំពុងរត់លើគោលដៅពិត។
`--arch amd64`	ត្រូវនឹង target។ បើ container ARM លទ្ធផលមិនត្រឹមត្រូវ។
`proof.json`	ភស្តុតាងក្នុង container។ មិនមែនភស្តុតាងលើ target ពិតទេ។
`profile.json`	Profile ដែលបានមក។ ត្រូវផ្គូផ្គង version និង SAPI មុនប្រើ។

ចំណុចត្រួតពិនិត្យ: តើ proof file ក្នុង container បញ្ជាក់ថា target ពិតអាច exploit បានទេ?

ទេ។ វាបញ្ជាក់តែថា exploit ដំណើរការក្នុងបរិស្ថាន calibration ដែលអ្នកបានសាងសង់។ លើ target ពិត ត្រូវមាន preconditions ទាំងអស់ និងការអនុញ្ញាតដាច់ដោយឡែក។

ជំពូក 9

ការពារ និងជួសជុល

គោលបំណងនៃជំពូកនេះ

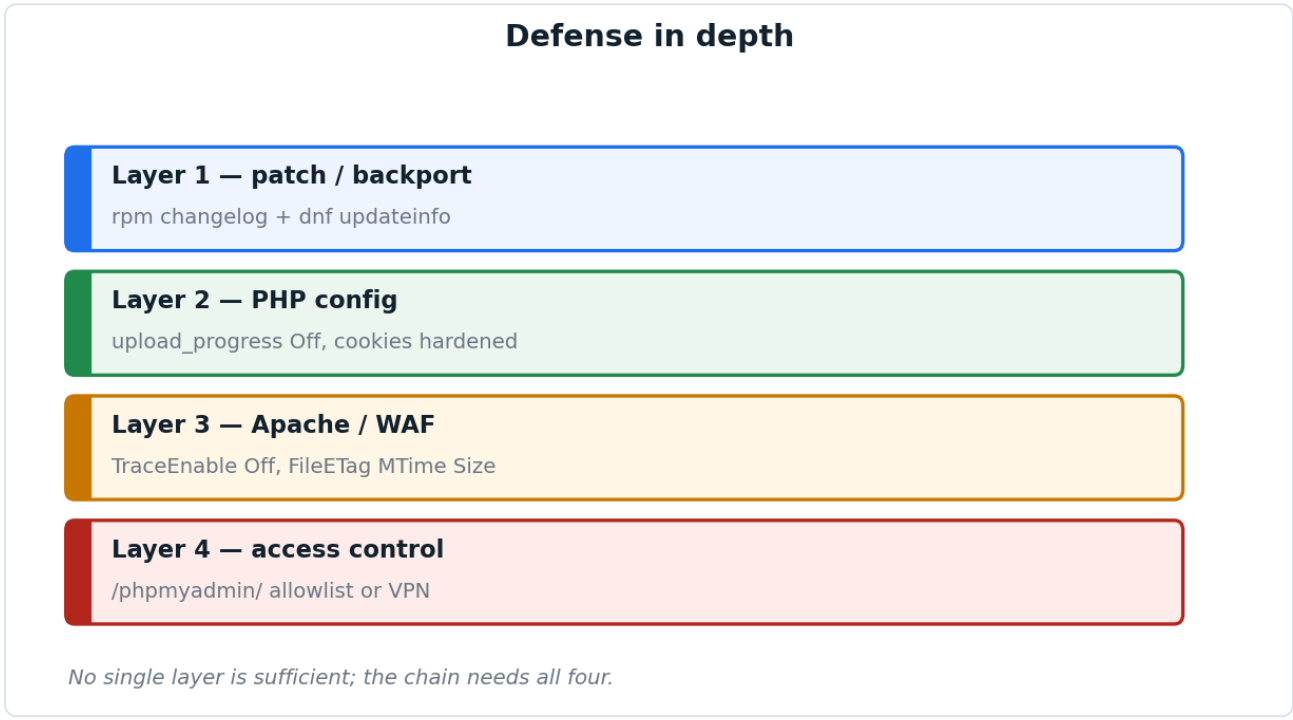
- ផ្ទៀងផ្ទាត់ backport ដោយ rpm changelog និងឈប់ បើ patched។
- អនុវត្ត hardening លើ PHP និង Apache។
- កំណត់ access ទៅ /phpmyadmin/ និងបិទផ្លូវចូលដែលមិនចាំបាច់។

ជំហានដំបូងគឺផ្ទៀងផ្ទាត់ថា distro បាន backport CVE ទាំងពីរ ឬអត់។ បើ rpm changelog មាន CVE-2026-109618 និង CVE-2026-109619 នោះអ្នកអាចឈប់ ហើយកត់ត្រាភស្តុតាង។ បើគ្មាន ត្រូវ upgrade ទៅ release ដែល patched តាម branch៖ 8.2 → 8.2.35, 8.3 → 8.3.36, 8.4 → 8.4.27, 8.5 → 8.5.12 ដោយសាកល្បងលើ staging មុន។ Branch EOL (7.4/8.0/8.1) គ្មាន release ថ្មីទេ ដូច្នេះការពារដោយ config រួចរៀបចំផែនការផ្លាស់ stream។

ការកាត់បន្ថយហានិភ័យជាបន្ទាន់រួមមាន៖ បិទ session.upload_progress បើមិនចាំបាច់, បិទ expose_php, កំណត់ cookie ឱ្យមាន HttpOnly និង Secure, បិទ TraceEnable, កំណត់ FileETag MTime Size និងបន្ថែម header ការពារ។ ទន្ទឹមគ្នា ត្រូវកំណត់ admin panel (ឧ. /phpmyadmin/, /admin/) ឱ្យចូលបានតែពី IP ដែលអនុញ្ញាត ឬតាម VPN។

> **គន្លឹះ**
> បើមិនអាច upgrade PHP បានភ្លាមៗ សូមបិទ access ទៅ admin panel និងបិទ session.upload_progress ជាមុន។ ទាំងពីរនេះកាត់បន្ថយផ្លូវវាយប្រហារបានភ្លាមៗ ទោះមិនមែនជាការជួសជុលពេញលេញក៏ដោយ។

ការពារជាស្រទាប់មានន័យថា បើស្រទាប់មួយខូច ស្រទាប់ផ្សេងនៅតែកាត់បន្ថយផលប៉ះពាល់។ ការអាប់ដេត package, ការកំណត់ PHP ត្រឹមត្រូវ, Apache hardening និង access control គួរធ្វើជាមួយគ្នា។ បន្ទាប់ពីកែ ត្រូវផ្ទៀងផ្ទាត់ដោយ curl, httpd -t និង php -i ម្តងទៀត។



រូបភាព 9.1 — ស្រទាប់ការពារ 4 ស្រទាប់ ចាប់ពី patch ដល់ access control

snippet — 08_harden_apache.conf

```
# បិទ information leak និង TRACE ក្នុង Apache។
# មើល httpd -t និង curl -I បន្ទាប់ពី reload។ ហានិភ័យ៖ ទាប។
TraceEnable Off
ServerTokens Prod
ServerSignature Off
FileETag MTime Size

Header always set X-Content-Type-Options "nosniff"
Header always set X-Frame-Options "SAMEORIGIN"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always set Permissions-Policy "geolocation=(), microphone=(), camera=()"
```

snippet — 09_php_ini_hardening.ini

```
;បិទ session upload progress និងកាត់បន្ថយ information leak។
;មើល php -i និងបញ្ជាក់ថា upload_progress.enabled = Off។ ហានិភ័យ៖ ទាប។
session.upload_progress.enabled = Off
expose_php = Off
session.cookie_httponly = 1
session.cookie_secure = 1
session.cookie_samesite = Lax
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`TraceEnable Off`	បិទ HTTP TRACE។ ជា hardening តូច ប៉ុន្តែគ្មានហេតុផលបើក។
`FileETag MTime Size`	ដក inode ចេញពី ETag ដើម្បីកាត់បន្ថយ information leak។
`session.upload_progress.enabled=Off`	បិទផ្លូវដែលខ្សែសង្វាក់ត្រូវការ។ ត្រូវផ្ទៀងផ្ទាត់ថា app មិនខូច។
`Require ip`	កំណត់ admin path ឱ្យចូលបានតែពី IP ដែលអនុញ្ញាត ឬ VPN។

ចំណុចត្រួតពិនិត្យ: តើការបិទ session.upload_progress អាចធ្វើឱ្យ Nextcloud ខូចទេ?

អាចមានឥទ្ធិពលលើ progress bar នៃ upload ធំៗ។ ត្រូវសាកល្បងលើ staging និងតាមដាន log។ បើ app ត្រូវការ feature នេះ ត្រូវជ្រើសរវាងការបិទ និងការកំណត់ access control បន្ថែម។

10 ជំពូក Checklist ចុងក្រោយ

គោលបំណងនៃជំពូកនេះ

- អនុវត្តលំដាប់ជួសជុលដោយចម្លងពាក្យបញ្ជាបាន។
- ប្រើដើមឈើសម្រេចចិត្តដើម្បីឈប់ ឬបន្ត upgrade។
- កត់ត្រាភស្តុតាង និងរាយការណ៍ដោយមិនលើសពីអ្វីដែលបានបញ្ជាក់។

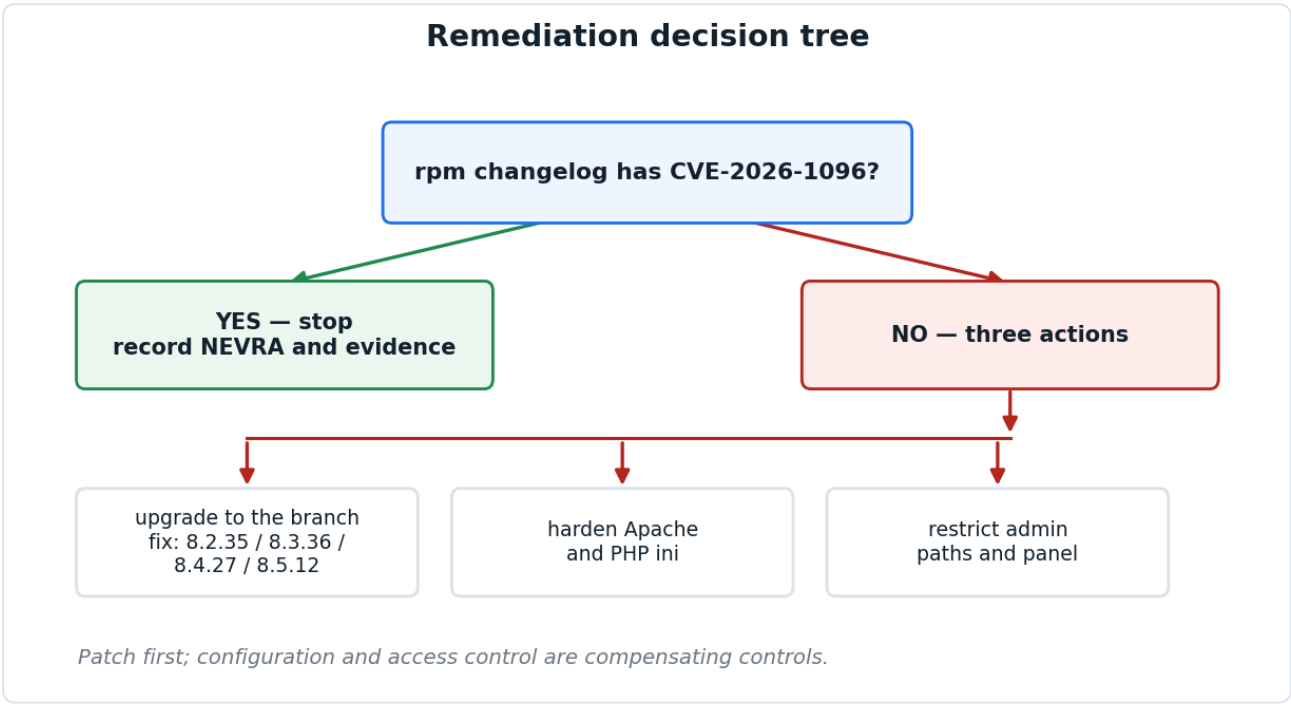
Checklist នេះត្រូវអនុវត្តតាមលំដាប់។ ទីមួយ ផ្ទៀងផ្ទាត់ rpm changelog និង dnf updateinfo។ បើ patched ឈប់ ហើយកត់ត្រា NEVRA។ បើមិន patched ត្រូវ backup, កំណត់ maintenance window, upgrade ទៅ release ដែល patched តាម branch (8.2.35 · 8.3.36 · 8.4.27 · 8.5.12), restart httpd និងផ្ទៀងផ្ទាត់សេវា។ បើ branch នោះ EOL ហើយគ្មាន release សូមបិទ session.upload_progress ជាការពារបណ្តោះអាសន្ន រួចដាក់ផែនការផ្លាស់ stream។ បន្ទាប់មក កំណត់ admin path និងអនុវត្ត hardening ផ្សេងទៀត។

ដើមឈើសម្រេចចិត្តខាងក្រោមជួយកាត់បន្ថយការសម្រេចចិត្តដែលច្របូកច្របល់។ បើ changelog មាន CVE សូមឈប់។ បើគ្មាន សូមអនុវត្ត 3 សកម្មភាព៖ upgrade PHP, harden Apache/PHP ini និង restrict phpMyAdmin។ ការធ្វើទាំងបីជាមួយគ្នាផ្តល់ការការពារល្អជាងការធ្វើតែមួយ។

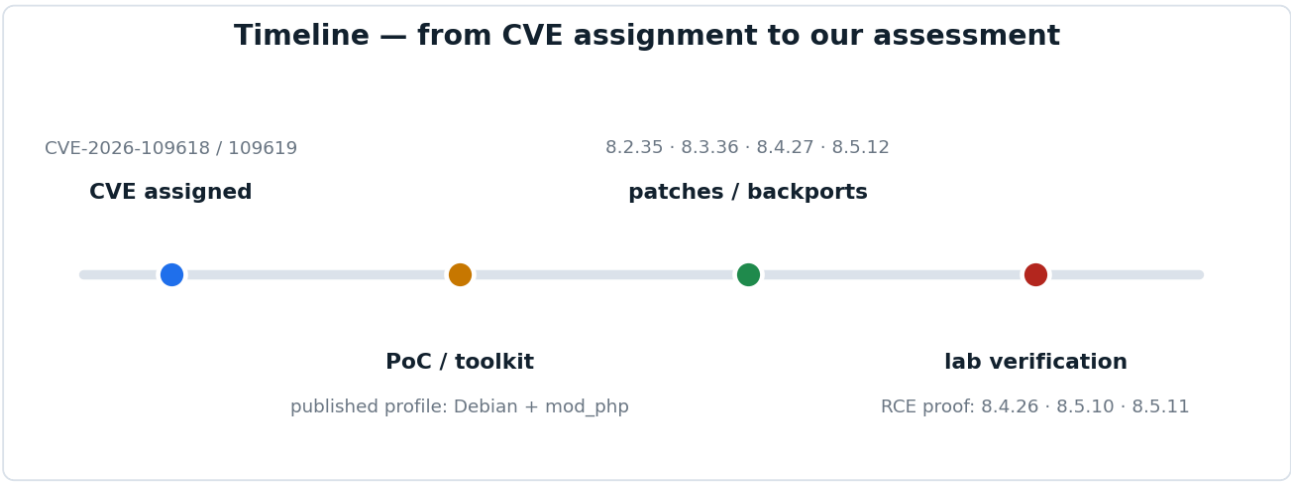
ចំណុចសំខាន់

បើគ្មាន LFI ខ្សែសង្វាក់ឈប់ ហើយ RCE មិនត្រូវបានបញ្ជាក់។ របាយការណ៍ត្រូវនិយាយបែបនេះដោយត្រង់ៗ មិនត្រូវបំផ្លើសពីភស្តុតាងទេ។

បន្ទាប់ពីជួសជុល ត្រូវរត់ការផ្ទៀងផ្ទាត់ឡើងវិញ៖ curl ទៅ hostname, httpd -t, php -v, php -i និង occ status បើមាន Nextcloud។ កត់ត្រាលទ្ធផលទាំងអស់ក្នុង ticket ឬរបាយការណ៍។ ការបញ្ជាក់ថា patched ដោយគ្មាន changelog ឬ erratum គឺមិនគ្រប់គ្រាន់ទេ។



រូបភាព 10.1 — ដើមឈើសម្រេចចិត្តជួសជុល៖ ឈប់ បើ changelog មាន CVE; បើគ្មាន ធ្វើ 3 សកម្មភាព



រូបភាព 10.2 — បន្ទាត់ពេលវេលាពីការចាត់ CVE ដល់ការវាយតម្លៃរបស់យើង

snippet — 10_final_checklist.sh

```
#!/usr/bin/env bash
# Checklist តាមលំដាប់៖ ផ្ទៀងផ្ទាត់ backport មុន បន្ទាប់មក upgrade ទៅ release ដែល patched តាម branch។
# មើលលទ្ធផលនៅរាល់ជំហាន; កុំរត់ក្នុងម៉ោងធ្វើការ។ ហានិភ័យ៖ មធ្យម។
set -euo pipefail

TARGET="${1:-http://127.0.0.1:8100}"

echo "== 1. ផ្ទៀងផ្ទាត់ changelog =="
if rpm -q --changelog php 2>/dev/null | grep -qi 'CVE-2026-109618\|CVE-2026-109619'; then
  echo "PHP មាន backport សម្រាប់ CVE ទាំងពីរ — ឈប់នៅទីនេះ ហើយកត់ត្រា NEVRA។"
  rpm -q php
  exit 0
fi

echo "== 2. មិនឃើញ backport — upgrade ទៅ release តាម branch =="
cat <<'EOF'
8.2 → 8.2.35  8.3 → 8.3.36  8.4 → 8.4.27  8.5 → 8.5.12
7.4 / 8.0 / 8.1 គ្មាន release ថ្មីទេ (EOL, មានតែ minimal source backport)។
សម្រាប់ branch EOL ការពារដោយ config៖ session.upload_progress.enabled=Off,
បិទ expose_php, កំណត់ផ្លូវ session read ហើយរៀបចំផែនការផ្លាស់ stream។
សាកល្បងលើ staging មុន — ការផ្លាស់ stream អាចធ្វើឱ្យ app ខូច។
EOF

read -r -p "វាយ YES ដើម្បីបន្តលើ staging: " ok
[ "$ok" = "YES" ] || exit 1

dnf module reset php -y
dnf module enable php:remi-8.3 -y
dnf update 'php*' --allowerasing -y
httpd -t
systemctl restart httpd
php -v

echo "== 3. ផ្ទៀងផ្ទាត់ក្រោយ upgrade =="
rpm -q --changelog php | grep -i 'CVE-2026-1096' || true
curl -skI "$TARGET/" | head
```

បន្ទាត់ / លទ្ធផល	អត្ថន័យ
`rpm -q --changelog php`	ជាភស្តុតាងដំបូង។ បើមាន CVE ឈប់ ហើយកត់ត្រា។
`dnf module switch`	ធ្វើតែលើ staging និងបន្ទាប់ពីផ្ទៀងផ្ទាត់ Nextcloud។
`systemctl restart httpd`	ធ្វើក្នុង maintenance window និងពិនិត្យ status។
`curl -I $TARGET`	ផ្ទៀងផ្ទាត់ថាសេវានៅដំណើរការ និង header ត្រូវបានកែ។ លើ lab គឺ http://127.0.0.1:8100។

ចំណុចត្រួតពិនិត្យ: តើពេលណាគួរឈប់រក exploit និងជួសជុល?

ឈប់ភ្លាម បើ assessment បញ្ជាក់ថាគ្មាន LFI ឬបើ changelog បញ្ជាក់ថា patched។ ការបន្តរក exploit បន្ថែមលើ production បង្កហានិភ័យ downtime ដោយគ្មានផលប្រយោជន៍។ ជួសជុល និងផ្ទៀងផ្ទាត់គឺជាជំហានត្រឹមត្រូវបំផុត។